

Hybrid SDP Bounding Procedure

F. Furini, E. Traversi

Abstract

The principal idea of this paper is to exploit Semidefinite Programming (SDP) relaxation within the framework provided by Mixed Integer Nonlinear Programming (MINLP) solvers when tackling Binary Quadratic Problems. We included the SDP relaxation in a state-of-the-art MINLP solver as an additional bounding technique and demonstrated that this idea could be computationally useful. The Quadratic Stable Set Problem is adopted as the case study. The tests indicate that the Hybrid SDP Bounding Procedure allows an average 50% cut of the overall computing time and a cut of more than one order of magnitude for the branching nodes.

Keywords: Binary Quadratic Problems, Semidefinite Relaxation, Branch and Cut, Quadratic Stable Set Problem.

1 Introduction

There are two main classical approaches present in the literature for solving Binary Quadratic Problems (BQP). The first one is directly using a Mixed Integer Nonlinear Programming (MINLP) solver to tackle a mathematical formulation (possibly linearizing the quadratic terms). The second approach uses Branch and Bound techniques which rely on the Semidefinite Programming (SDP) relaxation. The advantage of using MINLP solvers is that they have been strongly developed for decades. To mention just a few examples, we can cite some commercial software like BARON [1], CPLEX [2] and Gurobi [3]; as well as non commercial, for instance Bonmin [4]. They rely on sophisticated Branch and Cut (BC) algorithms based on a smart implicit enumeration of the branching tree. The bounding procedure typically makes use of Continuous Relaxation (CR) in order to prune the branching nodes. This relaxation can be efficiently computed but often, to be effective, it must be strengthened by adding families of valid inequalities. The second approach relies instead on the SDP relaxation. This relaxation is typically stronger than CR but generally it is computationally heavy. To the best of our knowledge, there are not many generic SDP-based solvers, BiqCrunch [5] to name one. On the other hand, there are many problem-oriented SDP algorithms, and we refer the interested reader for instance to [6], [7] or [8]. Within this context, we mention some works (see for example [9]), where the authors compare SDP relaxation and CR proving that stronger bounds can be achieved.

The principal contribution of this paper is to combine the strengths of both approaches, exploiting the SDP relaxation when enhancing the pruning strategies and thus the overall performances of a MINLP solver. To summarize, our goal boils down to addressing the following research questions:

- Is it worth exploiting the strength of bounds provided by SDP relaxation within a well tuned BC framework?

- Which mathematical formulation will profit more from the addition of the SDP bounding techniques?
- Which SDP relaxation has the best trade-off between the time saved by pruning nodes and the time needed for computing the SDP relaxation?
- What additional policies are necessary in order to improve the overall computational performances?

The presentation of the theoretical part of the paper is done for the generic case of the BQP. After each technique is presented, we apply it to the specific case of the Quadratic Stable Set Problem (QSSP). Recalling that a stable set in a graph is a subset of vertices such that for every two vertices selected in the solution, there is no edge connecting the two, the formal definition of the QSSP is the following: given an undirected graph $G = (V, E)$, with V ($n = |V|$) the set of vertices and E the set of edges, a vector of linear profit $L \in \mathbb{R}^n$ and a symmetric matrix of quadratic profit $Q \in \mathbb{R}^{n \times n}$ (possibly negative), the Quadratic Stable Set Problem (QSSP) searches for a stable set of G with maximum profit. In other words, if vertices i and j are in the solution, not only the linear profits are collected but also an additional profit equal to Q_{ij} . This quadratic counterpart of the Linear Stable Set Problem has not received much attention in the literature (we refer the interested reader for the linear case to [10] and [11]). Furthermore, to the best of our knowledge, the only papers that address the QSSP are [12] and [13]. In these works it appears as the sub-problem of a Column Generation algorithm and little computational analysis is presented.

2 Different Mathematical Formulations

The first step of the present work is to introduce the mathematical formulations that can be handled by a generic MINLP solver, i.e. the Quadratic Formulation and the Linear Formulation.

2.1 Quadratic Formulation

The Quadratic Formulation (QF) of the BQP, with n variables and p constraints, is defined as follows:

$$\begin{aligned}
 (\text{QF}) \quad & \max \sum_{i=1}^n \sum_{j=1}^n Q_{ij} x_i x_j + \sum_{i=1}^n L_i x_i \\
 & x \in K \\
 & x \in \{0, 1\}^n,
 \end{aligned}$$

with $Q \in \mathbb{R}^{n \times n}$, $L \in \mathbb{R}^n$, $K = \{x \in \mathbb{R}^n : Ax \geq b\}$, $A \in \mathbb{R}^{p \times n}$ and $b \in \mathbb{R}^p$. Q is a generic symmetric matrix, not restricted to being convex.

In the case of QSSP on a given undirected graph $G = (V, E)$, with V the set of vertices and E the set of edges, we have:

$$K = \{x \in \mathbb{R}^n : x_i + x_j \leq 1, \forall \{i, j\} \in E\}.$$

In order to solve the QF, many generic MINLP solvers are available. In addition, other solvers, explicitly defined for BQP can be used, such as Cplex and GloMIQO.

2.2 Linear Formulation

Another option for modelling BQP is to linearise the quadratic terms (we refer the interested reader to [14]) and obtain the following Linear Formulation (LF):

$$\begin{aligned}
 \text{(LF)} \quad \max \quad & \sum_{i=1}^n \sum_{j=i}^n Q_{ij} y_{ij} + \sum_{i=1}^n L_i x_i \\
 & \left. \begin{aligned} y_{ij} &\leq x_i \\ y_{ij} &\leq x_j \\ y_{ij} &\geq x_i + x_j - 1 \\ y_{ij} &\geq 0 \end{aligned} \right\} i, j = 1, \dots, n \\
 & x \in K \\
 & x \in \{0, 1\}^n.
 \end{aligned} \tag{1}$$

This linearization increases the size of the problem adding at most n^2 non-negative variables and $3n^2$ constraints.

3 SDP formulation

BQP can also be formulated as follows:

$$\begin{aligned}
 \max \quad & \langle \tilde{Q}, Y \rangle \\
 Y = & \begin{pmatrix} 1 \\ \bar{x} \end{pmatrix} \begin{pmatrix} 1 \\ \bar{x} \end{pmatrix}^\top
 \end{aligned} \tag{2}$$

$$\bar{x} \in \bar{K} \tag{3}$$

$$\bar{x} \in \{-1, 1\}^n \tag{4}$$

obtained after applying the linear transformation $\bar{x}_i = 2x_i - 1$, imposing

$$\tilde{Q} = \begin{pmatrix} \frac{1}{2} \sum_{j=1}^n L_i + \frac{1}{4} \sum_{i=1}^n \sum_{j=1}^n Q_{ij} & \frac{1}{4} (L + \frac{1}{2} \sum_{j=1}^n Q_j)^\top \\ \frac{1}{4} (L + \frac{1}{2} \sum_{j=1}^n Q_j) & \frac{1}{4} Q \end{pmatrix}$$

where Q_j is the j -th column of Q , and K is modified to $\bar{K}$ accordingly.

For the QSSP we have:

$$\bar{K} = \{x \in R^n : x_i + x_j \leq 0, \forall \{i, j\} \in E\} .$$

Alternatively, we can exploit the fact that the quadratic constraints $x_i x_j = 0$ for every edge $\{i, j\} \in E$ (valid for the stable set) becomes linear when rewritten in the Y space, we can hence instead of $\bar{K}$ add the following set of valid equations:

$$Y_{ij} + Y_{0i} + Y_{0j} + 1 = 0, \forall \{i, j\} \in E . \quad (5)$$

Constraints (2) and (4) together can be rewritten as $Y \succeq 0$, $\text{diag}(Y) = e$ and $\text{rank}(Y) = 1$, leading to the following equivalent formulation:

$$\begin{aligned} \max \quad & \langle \tilde{Q}, Y \rangle \\ & Y \succeq 0 \\ & \text{rank}(Y) = 1 \end{aligned} \quad (6)$$

$$\begin{aligned} & \text{diag}(Y) = e \\ & Y_0 \in \bar{K} \end{aligned} \quad (7)$$

with e being the all-ones vector and Y_0 being the first row of Y without the first element (note that $Y_0 = \bar{x}$). To the best of our knowledge, few solvers are able to deal directly with this formulation, BiqCrunch is one of them. Normally these solvers rely on solving special kinds of SDP relaxations. By relaxing the rank constraints (6) we obtain the classic SDP relaxation which provides valid bounds for all the BQP formulations. Note that this relaxation can be weakened by eliminating constraints (7) or strengthened by substituting them by the equations (5). Moreover, any valid inequalities for the LF, for the convex hull of $\bar{K}$ and for the convex hull of their intersection can be added in order to improve the bound further. An important class of valid inequalities for the SDP relaxation are the so-called triangle inequalities, see for example in [8], where the authors suggest how to carefully separate them and keep the overall computational time under control. Finally, in order to tackle the SDP relaxation, we recall that there are different solvers available in the literature, for instance [15] and [16].

4 Hybrid Bounding Procedure

The Hybrid Bounding Procedure is the idea of mixing two different relaxations, i.e. the basic relaxation used by a generic BQP solver and the SDP relaxation. As previously underlined there are many different possible ways of deriving an SDP relaxation for a BQP. In the following we will define with Ψ a generic version of the SDP relaxation. Furthermore we introduce another function called Ω which controls the fact of performing function Ψ in a specific node or not.

This is done because potentially the computation of Ψ can be time-consuming and hence it has to be used ideally only when worth it, in other words, when it prunes the node. Accordingly we need two functions available at each node of the BC tree:

- Function Ψ is the SDP relaxation that takes as input Q , L and a partial fixing of the variables and returns a bound (UB^{SDP}) on the original objective function.
- Function Ω is an oracle that takes as input all the information about the current node and returns a binary variable indicating whether Ψ should be used or not.

The node processing is represented in Algorithm 1, where LB is the incumbent best feasible solution.

Input: best incumbent solution of value LB and current variable fixing.

Output: 1 if the current node can be fathomed, 0 otherwise.

Solve the continuous relaxation and get the bound UB ;

if ($UB \leq LB$) **then return** 1.

$OK \leftarrow \Omega$.

if ($OK = \text{FALSE}$) **then return** 0.

solve Ψ and get the bound UB^{SDP} .

if ($UB^{SDP} \leq LB$) **then return** 1.

else return 0.

Algorithm 1: Processing at each decision node.

In the following computational section we measure the effectiveness of different options of Ψ and Ω in order to improve the overall efficiency.

5 Computational Experiments

The experiments are divided into two steps: first we start with a wide test-bed of instances of small sizes in order to test different possible options and strategies for the Hybrid SDP bounding procedure described in Section 4, once the best settings have been identified, we test bigger instances to evaluate the practical impact of our procedure. All algorithms were coded in C, and run on a PC with an Intel(R) Core2 Duo CPU E6550 at 2.33GHz and 2 GB RAM memory, under Linux Ubuntu 12 64-bit. The optimization software used in our test was Cplex 12.4 single thread. The SDP relaxation solver used is CSDP (described in [15]) and it was inserted in the Cplex framework using the `callBack` functions of the `Callable Library` of Cplex. Moreover, we stop the SDP solver as soon as we find a dual bound able to prune the current node, checking the corresponding dual feasibility.

5.1 Testbed description

As a first test-bed we randomly generated a set of QSSP instances; the goal was to have a statistical relevance (several instances with the same features) and to have the complete control of the characteristics of the different classes of instance proposed. The instance generator produces random graphs according to the desired number of vertices n and density μ (which implies a number of edges equal to $\lfloor \mu \times \frac{n(n-1)}{200} \rfloor$). The linear and quadratic profits take a uniformly

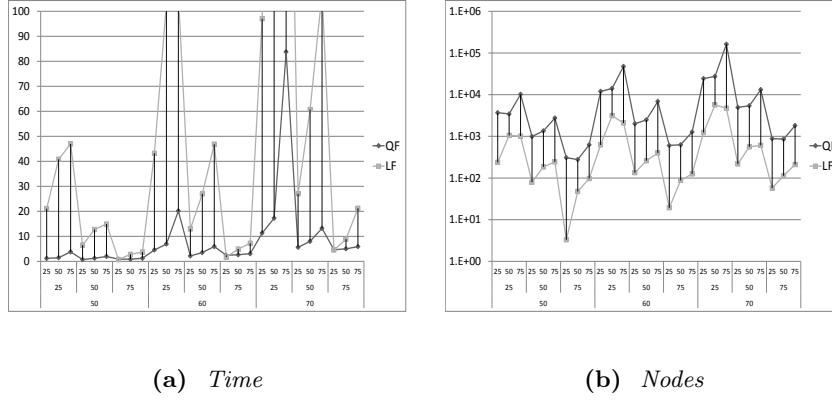


Figure 1: Performance comparison of different mathematical formulations (QF-LF)

random integer value in the interval $[-100, 100]$, a third parameter ν represents the percentage of positive profits. We generated 27 classes of instances by considering all combinations of:

- number of vertices: $n \in \{50, 60, 70\}$;
- density of edges: $\mu \in \{25\%, 50\%, 75\%\}$;
- percentage of positive costs: $\nu \in \{25\%, 50\%, 75\%\}$.

In addition we created 10 instances for each class using different random seeds, thus obtaining in total 270 QSSP instances. As a second test-bed we focus on instances with 100 vertices with the same range of densities and percentages of positive costs. The whole set of instances is available upon request to the authors. In the next sections we discuss the computational outcome of the experiments.

5.2 Identifying the best mathematical formulations

The goal of this section is to computationally evaluate the different formulations, i.e. QF or LF. In order to do that, we used Cplex with default parameter settings. In Figure 1(a) and 1(b) the computing time (seconds) and number of nodes (logarithmic scale) required for the optimization are reported, dividing the instances (from bottom to top) by vertex number (n), density (μ) and percentage of positive profits (ν). As far as the computing time is concerned, the best mathematical formulation is always QF. It is interesting to stress that the behaviour in terms of number of nodes is exactly the opposite: QF performs a number of nodes on average which is at least one order of magnitude bigger than LF. The tests also confirm the tendency for instances with low density to be more difficult than instances with high density. On the other hand, instances with a high percentage of positive profits ν are more difficult than the ones with a low value. These experiments allow us to conclude that QF is better than LF. Moreover, the fact that QF explores a larger amount of nodes makes it a more promising candidate for testing the addition of a second bounding procedure. For these reasons, in the rest of the tests we will focus only on QF.

n	Ratios						Times					
	QF	LF	LF _c	Uncon	Con1	Con2	QF	LF	LF _c	Uncon	Con1	Con2
50	4.0	8.9	2.1	7.5	4.8	1.1	0.1	0.0	7.5	0.0	1.8	1.3
60	4.8	11.8	2.6	9.7	6.0	1.1	0.2	0.1	14.7	0.0	4.6	3.1
70	5.2	14.3	2.9	11.1	6.9	1.1	0.2	0.2	29.2	0.1	10.0	6.9
	4.7	11.7	2.5	9.4	5.9	1.1	0.2	0.1	17.1	0.0	5.5	3.8

Table 1: Bound Comparison

5.3 Different strategies for Ψ

We propose the following three different options for Ψ :

- **Unconstrained**: SDP relaxation without constraints.
- **Constrained**: SDP relaxation with the family of constraints (7).
- **Constrained2**: SDP relaxation with the family of constraints (5).

In order to access the performance of the different options, six different bounds are computed and compared in Table 1. **QF** is the root bound of the Cplex BQP solver, **LF** is the root bound of the continuous relaxation of LF and **LF_c** is like LF but with the addition of the cuts separated by Cplex in the root node. Finally **Uncon**, **Con1** and **Con2** are respectively the bounds of the three different strategies (Ψ). For each option we provide two values: the ratio between the bound obtained and the optimal solution (where a ratio of 1 indicates that the relaxation has no gap with the optimal solution) and the average time needed to compute it. The information concerning **QF** and **LF_c** confirms the results obtained in Section 5.2: a BC based on LF is not competitive. It is also interesting to notice that **Unconstrained** presents a worse gap than **QF**, although this does not imply that the addition of **Unconstrained** is not helpful because the SDP relaxation is computationally sensitive to variable fixing during the BC tree explorations and the bound provided becomes stronger in lower levels of the BC tree. If we consider the SDP based bounds we see that **Constrained1** is dominated by **Constrained2** and that there is an interesting trade-off between **Constrained2** and **Unconstrained**. Between the three configurations proposed we hence decided to discard **Constrained** and to focus on **Unconstrained** and **Constrained2**.

5.4 Different strategies for the Oracle Ω

Ideally the SDP relaxation should be performed only in the cases in which it helps in pruning. To cope with this problem, we tried seven different strategies for the Oracle Ω . The strategies used are:

- **1 Always**. The SDP relaxation Ψ is triggered at each node of the BC tree.
- **2 OnOne**. Ψ is triggered every time we branch on one.
- **3 UnderAverage**. Ψ is triggered if the current integrality gap is lower than the average integrality gap of the nodes explored so far.
- **4 OverAverage**. Ψ is triggered if the current integrality gap is bigger than the average.

- 5 **SmallGap**. Ψ is triggered only when the integrality gap is within $[0\%, 5\%]$.
- 6 **MediumGap**. Ψ is triggered only when the integrality gap is within $[5\%, 30\%]$.
- 7 **Random**. Ψ is triggered with a random 50% probability.

Strategy **OnOne** exploits the fact that usually when we branch on one the solution in the child nodes will probably change more significantly than when we branch on zero, increasing the probability of pruning. Strategies **OverAverage** and **MediumGap** tend to prune the branch and bound tree at the first levels and they are effective in the case in which the SDP relaxation is much stronger than the continuous relaxation, and/or they tend not to prune in the same points of the branching tree. Strategies **UnderAverage** and **SmallGap** tend to prune the nodes at the final levels and are effective in the cases in which the LP relaxation and the SDP relaxation are strong in the same nodes. Finally, strategies **Always** and **Random** serve as terms of comparison.

5.5 Identifying the best Ω

Let QF be the basic formulations and $QF(\Psi, \Omega)$ be the same formulation using a given couple of SDP relaxation (Ψ) and Oracle strategy (Ω). Four performance indices are collected to assess the computational impact:

$$\delta = \frac{100 \cdot \text{nod}_{QF(\Psi, \Omega)}}{\text{nod}_{QF}}, \quad \tau = \frac{100 \cdot t_{QF(\Psi, \Omega)}}{t_{QF}}, \quad \pi = \frac{100 \cdot \hat{t}_{QF(\Psi, \Omega)}}{t_{QF}}, \quad \beta = \frac{100 \cdot t_{SDP}}{t_{total}}$$

with t being the total optimization time and nod the number of nodes in the BC tree. Time $\hat{t}$ represents the “useful” computation time, in other words the total time minus the time used for non-pruning SDP relaxation Ψ . t_{SDP} is the total time spent solving SDP in the bounding procedure. Values of δ and τ lower than one hundred correspond to an improvement in the performances of the BC. The percentage π gives what in statistics is called “gold standard”, in other words it gives an idea about what the results could be in presence of an “exact” Ω . β is useful in order to establish how much an increase in the speed of the SDP solver would affect the overall performances.

In Figure 2 we compare how these indices change with the increase of the instance size when we use **Constrained2** and all the Ω strategies described. In particular, from Figure 2(a) we see that the Strategies **OnOne**, **SmallGap** and **MediumGap** are promising because of the decreasing trend of τ that goes below the threshold value of 100. The **UnderAverage** trend seems roughly constant and the remaining three strategies seem unpromising. From Figure 2(b) we see that with **SmallGap** and **MediumGap** the pruning index δ is high, for these strategies the SDP relaxation is not often computed. The overall good performance of **OnOne** can be explained by comparing the performance indices δ and β in Figures 2(b) and 2(d): on one side the trend of δ is going below 10, in other words ensuring a decrease of the number of processed BC nodes greater than one order of magnitude; on the other, the fraction of time spent in solving the SDP (β) is significantly lower when compared to other strategies (but still relevant in comparison to **SmallGap** and **MediumGap**). Those two points together make **OnOne** the best candidate for Ω . Finally, as one could expect, the majority of the trends of the “gold standard” π (represented in

Figure 2(c)) present values lower than one hundred.

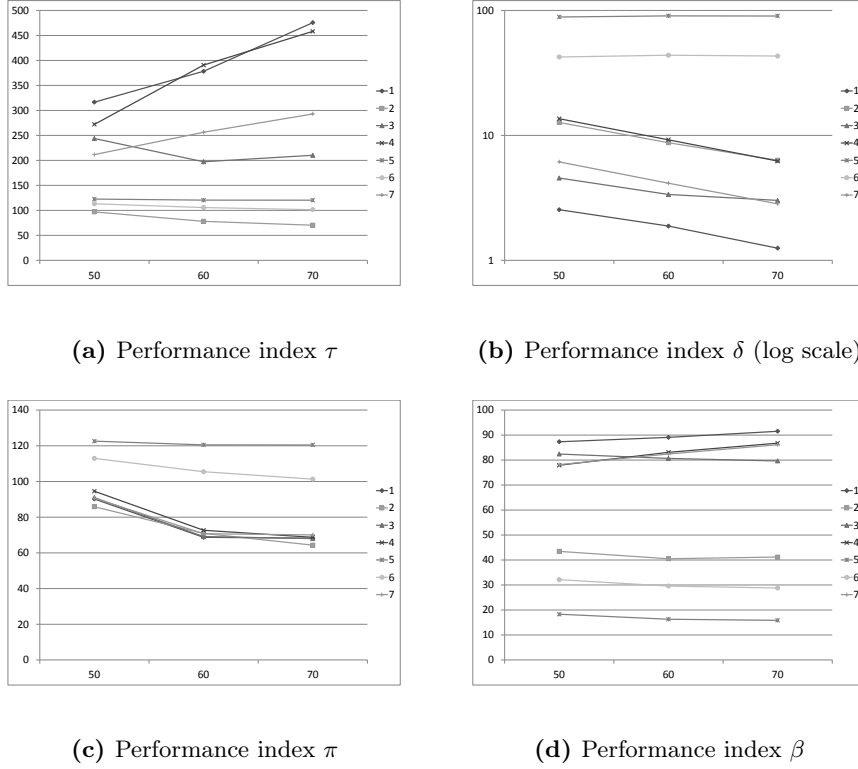


Figure 2: Comparison of the different strategies (Ω)

In order to study the interaction between Ψ and Ω we introduce Table 2. The upper part of the table concerns the **Unconstrained** option and the lower part the **Constrained2** option. The structure of both parts is identical: vertically the instances are divided first by density and then by percentage of positive weights and horizontally, the values of δ and τ concerning the 7 different strategies are reported. Each entry in the table is the average value over 10 instances of identical features. Each entry of τ is in bold if the index is less than one hundred (i.e. if the addition of (Ψ, Ω) improved the performances). The instances considered have 70 vertices. Concerning the node reduction, strategy **Always** gives the best insight about the maximum reduction achievable. With respect to the SDP strategy Ψ , as expected, **Constrained2** dominates **Unconstrained** node-wise, leading to a reduction of almost two orders of magnitude in terms of nodes explored when the edge constraints are also taken into account. If we observe the same entries for the **Unconstrained** mode we see a decrease of slightly less than one order of magnitude. If we consider the time index τ , the best behaviour is given by the couple (**Constrained2**, **OnOne**), that guarantees on average an improvement of about 30% over the total running time.

5.6 Results for instances with 100 nodes

One main issue when dealing with a SDP-based approach is the scalability of the method. In order to answer to this question, we keep the best settings obtained from the first set of tests and we use them to solve bigger instances, i.e., strategy **Constrained2** (Ψ) and **OnOne** (Ω). We also decided to keep strategy **Always** (Ω) in the analysis for comparison. In Table 3

strategy			δ							τ						
			1	2	3	4	5	6	7	1	2	3	4	5	6	7
n	μ	ν	SDP Unconstrained													
70	25	25	1.5	8.3	1.6	9.6	83.7	20.4	4.7	78.6	65.3	65.9	63.4	144.4	99.5	61.3
		50	13.8	31.9	13.9	57.9	80.4	19.2	29.5	201.3	125.8	149.2	185.9	134.9	113.6	153.7
		75	21.9	43.0	22.8	83.6	79.4	29.2	44.7	227.3	131.2	159.3	201.9	124.1	136.8	177.8
50	25	25	4.0	15.4	4.1	17.6	96.4	50.7	9.9	96.9	68.1	82.1	93.9	102.5	88.2	83.5
		50	13.6	31.7	14.8	46.6	92.3	43.5	28.1	108.9	80.2	89.4	111.9	100.5	86.6	97.1
		75	16.4	36.7	16.9	56.3	90.0	47.2	33.4	105.0	74.2	80.2	116.2	97.8	80.5	93.3
75	25	25	10.0	23.3	14.3	31.1	97.1	63.0	20.7	111.6	90.7	95.7	113.7	99.9	96.3	103.0
		50	20.8	40.5	22.2	55.8	98.5	65.5	37.9	111.3	94.9	97.1	113.8	100.3	97.9	104.8
		75	18.7	40.0	20.6	57.2	94.6	66.6	36.5	109.6	89.4	91.3	114.3	99.5	95.7	101.7
avg.			13.4	30.1	14.6	46.2	90.2	45.0	27.3	127.8	91.1	101.1	123.9	111.5	99.5	108.5
n	μ	ν	SDP Constrained2													
70	25	25	0.4	2.1	0.7	3.8	83.7	20.4	1.1	120.3	65.8	109.9	76.4	186.9	174.0	83.4
		50	0.4	1.8	0.8	4.0	80.4	13.9	0.9	71.8	49.4	61.2	54.1	160.5	103.5	46.8
		75	0.1	0.4	0.2	1.1	79.9	17.7	0.3	16.6	13.4	16.1	15.4	133.4	76.1	13.4
50	25	25	1.2	6.4	2.7	5.8	96.4	50.7	3.0	765.7	95.6	404.3	623.8	103.4	94.3	446.8
		50	0.7	5.2	1.8	4.9	92.3	43.9	2.0	232.5	82.4	113.7	262.4	101.6	91.6	169.3
		75	0.5	3.1	1.4	3.5	90.0	47.0	1.3	207.0	55.2	97.4	213.5	98.1	82.0	146.4
75	25	25	2.9	15.4	8.7	10.1	97.1	63.0	6.0	1033.3	94.0	371.7	1139.7	99.9	96.8	655.1
		50	3.4	13.9	6.6	14.3	98.5	65.4	6.0	796.6	91.4	319.9	660.0	100.4	98.3	420.9
		75	1.7	8.6	4.3	8.8	94.6	66.6	5.0	1038.2	85.9	398.1	1079.1	99.6	95.8	654.8
avg.			1.3	6.3	3.0	6.2	90.3	43.2	2.8	475.8	70.4	210.3	458.3	120.4	101.4	293.0

Table 2: QF + SDP Unconstrained and SDP Constrained2 (Ψ)

we report the results concerning the four parameters introduced in Section 5.5 subdivided horizontally by μ and ν and vertically by Ω strategy. In Columns *avg. times* we report the average times (in seconds) needed for solving the instances without (column QF) and with (column QF(Ψ, Ω)) the additional bounding procedure **Constrained2** using **OnOne** as oracle. The strategy (**Constrained2**, **OnOne**) is able to reduce the overall computational time by 50%. If we consider only the non dense instances ($\mu = 25$) the overall computational time is reduced by 80%. The results concerning **Always** shows how the idea of running the additional bounding procedure without an oracle is useless in terms of overall computational time.

			δ		τ		π		β		<i>avg. time</i>	
<i>str</i>			<u>1</u>	<u>2</u>	<u>1</u>	<u>2</u>	<u>1</u>	<u>2</u>	<u>1</u>	<u>2</u>	<u>QF</u>	<u>QF(Ψ,Ω)</u>
$\overline{n}$	$\overline{\mu}$	$\overline{\nu}$										
100	25	25	0.1	0.7	89.5	25.0	13.9	18.3	99.0	95.5	345.4	85.2
		50	0.1	1.0	49.1	24.0	11.5	17.9	90.9	84.6	419.2	86.9
		75	0.0	0.2	5.0	3.1	1.3	2.4	90.1	86.4	4378.0	133.7
50	25	0.3	2.5	679.3	67.0	57.9	57.4	97.5	50.3	41.6	27.9	
	50	0.2	1.9	489.7	54.9	42.7	48.9	96.7	41.5	54.5	30.0	
	75	0.2	2.2	274.8	31.0	22.9	23.6	95.8	48.8	120.6	37.6	
75	25	1.5	7.2	3449.9	80.9	172.6	79.7	99.3	9.1	20.3	16.4	
	50	1.4	11.3	2490.0	83.1	164.6	79.7	98.9	9.2	22.7	18.8	
	75	0.9	6.4	2269.8	72.2	123.9	69.3	98.2	10.3	27.0	19.5	
avg.			0.5	3.7	1088.5	49.0	67.9	44.1	96.3	48.4	603.3	50.7

Table 3: QF + Constrained2 SDP (Ψ) - Instances with 100 nodes

6 Conclusions

In this paper we have explored the use of SDP-based bounding procedures within the BC framework provided by MINLP solvers. In order to do that, we performed an extensive computational analysis on the QSSP that allows us to conclude that the Hybrid SDP Bounding Procedure allows a noticeable reduction of computing time and BC nodes. The SDP bounds help in pruning but are heavy to compute, and thus in this optic we proposed different strategies in order to make the Hybrid bounding procedure more efficient. The addition of these strategies is crucial for the improvement of the performances over the standard BC.

Finally, the SDP relaxation can be used to enhance any of the essential ingredients of a generic purpose MINLP solver, which are: bounding techniques, primal heuristics and branching strategies. Specifically the information provided by the solution of the SDP relaxation at each branching node can be used either to derive alternative branching strategies, to compute different heuristic solutions or to strengthen LP relaxation for pruning the node. In this paper we have focused on this last aspect, deriving effective Hybrid Bounding Procedures. We leave the other two aspects for further development.

References

- [1] BARON, 2012. <http://archimedes.cheme.cmu.edu/?q=baron>.
- [2] Cplex, 2012. <http://www-01.ibm.com/software/integration/optimization/cplex-optimizer/>.
- [3] Gurobi, 2012. <http://www.gurobi.com/>.
- [4] Bonmin, 2012. <https://projects.coin-or.org/Bonmin>.
- [5] BiqCrunch, 2012. <http://www-lipn.univ-paris13.fr/BiqCrunch/>.
- [6] S. Burer and D. Vandenbussche. Globally solving box-constrained nonconvex quadratic programs with semidefinite-based finite branch-and-bound. *Comp. Optim. Appl.*, 43:181–195, 2009.
- [7] N. Krislock, J. Malick, and F. Roupin. Improved semidefinite bounding procedure for solving max-cut problems to optimality. *To appear in Math. Prog.*, 2013.
- [8] F. Rendl, G. Rinaldi, and A. Wiegele. Solving max-cut to optimality by intersecting semidefinite and polyhedral relaxations. *Math. Prog.*, 121:307–335, 2010.
- [9] K. M. Anstreicher. Semidefinite programming versus the reformulation-linearization technique for nonconvex quadratically constrained quadratic programming. *J. Global Optim.*, 43(2-3):471–484, 2009.
- [10] M. Giandomenico, A. N. Letchford, F. Rossi, and S. Smriglio. A new approach to the stable set problem based on ellipsoids. volume 6655 of *LNCS*, pages 223–234. Springer, 2011. ISBN 978-3-642-20806-5.
- [11] F. Mahdavi Pajouh, B. Balasundaram, and O. Prokopyev. On characterization of maximal independent sets via quadratic optimization. *J. Heuristics*, pages 1–16, 2011. ISSN 1381-1231.
- [12] B. Jaumard, O. Marcotte, and C. Meyer. *Estimation of the Quality of Cellular Networks Using Column Generation Techniques*. Cahiers du G  RAD. 1998. URL <http://books.google.it/books?id=DFwiMAEACAAJ>.
- [13] B. Jaumard, O. Marcotte, C. Meyer, and T. Vovor. Comparison of column generation models for channel assignment in cellular networks. *Discrete Appl. Math.*, 112(1–3):217 – 240, 2001.
- [14] H. D. Sherali and W. P. Adams. *A Reformulation-Linearization Technique for Solving Discrete and Continuous Nonconvex Problems*. Springer, 1998.

- [15] B. Borchers. CSDP, A C library for semidefinite programming. *Optim. Methods Softw.*, 11:613–623, 1999.
- [16] SeDuMi, 2012. <http://sedumi.ie.lehigh.edu/>.